

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

ОТЧЁТ

по учебной практике

**Тема: «Разработка интеллектуальной поисковой системы по
внутренней базе знаний университета»**

Выполнил: студент группы БПИ2403
Рыбаченок Вадим Александрович

Проверила: Мосева М. С.

Москва, 2026

1 Цель и задачи практики

Цель учебной практики — закрепление и углубление теоретических знаний, полученных при изучении дисциплин направления «Программная инженерия», через разработку полнофункционального программного продукта с использованием современных технологий. В рамках практики командой из четырёх студентов разрабатывалась интеллектуальная поисковая система по внутренней базе знаний университета — веб-приложение «Кампус», объединяющее загрузку учебных материалов, полнотекстовый поиск, учебные очереди на сдачу работ и безопасную интеграцию с сервисами МТУСИ.

В соответствии с распределением ролей на организационном собрании автор отчёта выполнял роль фронтенд-инженера (FE) и дополнительно реализовал серверный модуль интеграции с внешними сервисами МТУСИ. Индивидуальные задачи автора:

- спроектировать и реализовать пользовательский интерфейс веб-приложения для трёх ролей (студент, преподаватель, администратор);
- реализовать клиент-серверное взаимодействие с REST API и обработку авторизации по JWT;
- реализовать модуль поиска по базе знаний с подсветкой совпадений и пагинацией;
- реализовать подсистему учебных очередей на стороне клиента;
- обеспечить адаптивную вёрстку (320–1920 px) и работу в режиме Telegram Mini App;
- разработать серверный модуль интеграции с МТУСИ с шифрованием пользовательских токенов;
- подготовить конфигурацию контейнеризации фронтенда и техническую документацию;
- провести релиз проекта из ветки разработки в production.

2 Индивидуальное задание

Автор отвечал за фронтенд-часть системы, серверный модуль интеграции с МТУСИ, контейнеризацию фронтенда и релиз проекта. Ниже

приведён перечень пунктов технического задания, закрытых автором.

Таблица 1 — Реализованные требования технического задания

ID	Требование	Роль
FE-01	Каркас приложения на Vite, маршрутизация, HTTP-клиент с JWT-интерсерсом	FE
FE-02	Страница входа: демо-вход по ролям и форма MTUCI/TECH-токена	FE
FE-03	Загрузка документов Drag-and-Drop с прогрессом и статусами	FE
FE-04	Полнотекстовый поиск, подсветка совпадений, пагинация, пустые состояния	FE
FE-05	Дашборд с разными состояниями для трёх ролей	FE
FE-06	Список загруженных документов с метаданными и статусом	FE
FE-07	Пагинация результатов поиска (по 10 на страницу)	FE
FE-08	Сообщение при отсутствии результатов поиска	FE
FE-09	Адаптивная вёрстка 320–1920 px, режим Telegram Mini App	FE
BE-M1	Модель ExternalIntegration, шифрование токена (Fernet)	BE
BE-M2	Сервис MtuciTechService с мок-режимом	BE
BE-M3	Вход через МТУСИ и endpoints интеграции	BE
BE-M4	Нормализация расписания и учебных событий	BE
DO-02	Dockerfile фронтенда с отдачей статики через Nginx	DO
DO-04	Файл .env.example, вынесение секретов в переменные окружения	DO

3 Ход выполнения работы

Работа велась по методологии ветвления Git: каждая задача разрабатывалась в отдельной ветке и вливалась в ветку разработки через Pull Request (PR) после ревью. Всего автором подготовлено и слито 22 запроса на слияние; суммарный вклад составил свыше 14 700 добавленных строк кода. Ниже описаны основные группы выполненных работ.

3.1 Каркас и инфраструктура фронтенда

Реализован каркас приложения на React 18 и TypeScript со сборкой Vite. Настроена маршрутизация (React Router), защита приватных маршрутов (ProtectedRoute) и единый HTTP-клиент на Axios. Клиент автоматически подставляет заголовок Authorization с JWT-токеном и при ответе 401 сбрасывает токен и перенаправляет пользователя на страницу входа. Заведена структура каталогов api / components / pages / hooks / data, соответствующая требованиям к репозиторию из технического задания.

3.2 Аутентификация и роли

Реализована страница входа с двумя сценариями: демонстрационный вход по выбранной роли (студент, преподаватель, администратор) и вход по пользовательскому MTUCI/TECH-токену. Состояние авторизации вынесено в провайдер `useAuth`; для каждой роли настроен свой набор доступных действий и разделов интерфейса.

3.3 База знаний, загрузка и поиск документов

Реализованы страницы загрузки документов (Drag-and-Drop с индикацией прогресса и статусов «Загрузка», «Индексация», «Готово», «Ошибка») и базы знаний. Поиск запускается по кнопке «Найти» и по клавише Enter; результаты выводятся карточками с названием файла, номером страницы, фрагментом текста и оценкой релевантности. Совпадения подсвечиваются жёлтым фоном (компонент `Highlight`), реализована пагинация по 10 результатов и информативные пустые состояния.

3.4 Подсистема учебных очередей

Разработаны страницы списка очередей и деталей очереди, а также модальное окно создания очереди, в том числе кнопка «Создать очередь на защиту практики». Реализованы запись и выход из очереди, стабильная сквозная нумерация позиций, выставление оценок участникам преподавателем и управление составом очереди с подтверждающими диалогами.

3.5 Адаптивность и Telegram Mini App

Вёрстка адаптирована под ширину экранов от 320 до 1920 px. Приложение может работать как Telegram Mini App: при наличии Telegram WebApp SDK используются данные окружения Telegram (хук `useTelegram`), иначе включается обычный браузерный режим.

3.6 Серверный модуль интеграции с МТУСИ

На стороне бэкенда реализован модуль интеграции с внешними сервисами МТУСИ. Введена модель `ExternalIntegration` и миграция

Alembic; пользовательский токен шифруется алгоритмом Fernet и хранится в PostgreSQL только в зашифрованном виде, на фронтенд не возвращается. Разработан сервис MtuciTechService с мок-режимом для локальной разработки, endpoints входа и управления интеграцией, а также нормализация расписания и учебных событий.

3.7 Контейнеризация, документация и релиз

Подготовлен Dockerfile фронтенда с многоступенчатой сборкой и отдачей статики через Nginx, настроен healthcheck по IPv4, секреты вынесены в .env.example. Дополнена техническая документация (README) по запуску через Docker Compose и работе с MTUCI/TECH-токеном. Проведён релиз из ветки разработки dev в ветку production, а также реализованы скачивание файлов с ASCII-fallback имени, загрузка документов всеми ролями и отображение сведений о загрузившем.

Таблица 2 — Основные запросы на слияние (Pull Requests) автора

№ PR	Описание	Дата
46	FE-01 · Каркас Vite + Router + API-клиент	30.06.2026
47	FE-02 · LoginPage (демо-вход, MTUCI/TECH-токен)	30.06.2026
48	FE-05 · DashboardPage (3 роли)	30.06.2026
49	FE-03 · UploadDocumentsPage (Drag-and-Drop)	30.06.2026
50	FE-04 · KnowledgeBasePage (поиск, подсветка)	30.06.2026
51	FE-06 · QueuesPage + QueueDetailPage	30.06.2026
52	FE-P2 · Кнопка «Создать очередь на защиту практики»	30.06.2026
53	FE-08 · Адаптив 320–1920 px + Telegram Mini App	30.06.2026
54	FE-07 · ProfilePage + статус интеграции	30.06.2026
57	FE-FIX · Корректность очередей/документов, просмотр файлов	30.06.2026
59	FE-Q · Оценки в очереди, метаданные документов	30.06.2026
61	FE-Q2 · Стабильная нумерация очереди + оценка	30.06.2026
73	BE-M1 · ExternalIntegration + шифрование токена	04.07.2026
74	BE-M2 · MtuciTechService с мок-режимом	04.07.2026
75	BE-M3 · MTUCI-вход и endpoints интеграции	04.07.2026
76	BE-M4 · Нормализация расписания и сессии	04.07.2026
77	FE-INT · Синхронизация frontend API-контракта	04.07.2026
79	DevOps: Dockerfile, Nginx, .env.example, README	05.07.2026
80	Fix · Healthcheck фронтенда по IPv4	05.07.2026
82	Скачивание файлов, загрузка всем ролям, фиксы	05.07.2026

№ PR	Описание	Дата
84	Fix · ASCII-fallback имени, ограничение удаления	05.07.2026
83	Release · dev → prod	05.07.2026

4 Исходный код программы

Ниже приведены характерные фрагменты разработанного кода. Листинги оформлены шрифтом Consolas, 10 пунктов.

Листинг 1 — HTTP-клиент с JWT-интерсептором (frontend/src/api/client.ts)

```
export const api = axios.create({ baseURL: "/api/v1" });
api.interceptors.request.use((config) => {
  const token = getToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
api.interceptors.response.use(
  (response) => response,
  (error: AxiosError) => {
    if (error.response?.status === 401) {
      clearToken();
      window.dispatchEvent(new CustomEvent(AUTH_LOGOUT_EVENT));
    }
    return Promise.reject(error);
  }
);
```

Листинг 2 — Шифрование токенов интеграции (backend/app/services/token_crypto.py)

```
from cryptography.fernet import Fernet
from app.core.config import settings

def _fernet_key() -> bytes:
    if settings.token_encryption_key:
        return settings.token_encryption_key.encode()
    digest = hashlib.sha256(settings.jwt_secret_key.encode()).digest()
    return base64.urlsafe_b64encode(digest)

def encrypt_token(token: str) -> str:
    return Fernet(_fernet_key()).encrypt(token.encode()).decode()

def decrypt_token(encrypted_token: str) -> str:
    return Fernet(_fernet_key()).decrypt(encrypted_token.encode()).decode()
```

5 Заключение

В ходе учебной практики автором в составе команды разработана интеллектуальная поисковая система по внутренней базе знаний университета. Лично автором выполнены следующие задачи:

- разработан фронтенд веб-приложения на React и TypeScript для трёх ролей пользователей с маршрутизацией, защитой приватных страниц и единым HTTP-клиентом;
- реализованы загрузка документов, полнотекстовый поиск с подсветкой совпадений и пагинацией, подсистема учебных очередей;
- обеспечена адаптивная вёрстка 320–1920 px и работа приложения как Telegram Mini App;
- реализован серверный модуль интеграции с МТУСИ с шифрованием пользовательских токенов алгоритмом Fernet;
- подготовлены контейнеризация фронтенда, техническая документация и выполнен релиз проекта в production.

Поставленные индивидуальные задачи выполнены в полном объёме. В процессе работы закрепились навыки командной разработки с использованием системы контроля версий Git, ветвления и код-ревью через Pull Request, а также проектирования клиент-серверных веб-приложений. Итоговый программный продукт собирается и запускается одной командой через Docker Compose и соответствует требованиям технического задания.